BORDER OPTICS — Data Dictionary

Column definitions for the processed datasets in `data/processed/`, added as part of the project's reproducibility package alongside `requirements.txt` and the `src/` pipeline scripts.

`border_optics_master_villages.csv` / `border_optics_master_villages_with_distance.csv`

One row per successfully geocoded village (258 rows).

Column	Type	Description
`village_id`	int	Stable integer ID assigned during `merge_geocoded.py`; the join key used across every other processed file.
`village`	string	Village/habitation name (from `Habitation` in the raw state lists).
`district`	string	District name.
`block`	string	Administrative block. May be blank/"unresolved" for the 19 Pithoragarh (Uttarakhand) villages documented in BO_Development_Log.md, Entry 2.
`state`	string	One of Arunachal Pradesh, Sikkim, Uttarakhand, Himachal Pradesh.
`is_core_sample`	bool	`True` for Arunachal Pradesh / Uttarakhand (the 220-village core statistical sample from v2.1); `False` for Himachal Pradesh (7-village illustrative case study) and, from v2.1, for Sikkim (31 villages; no verified comparison villages). Until v2.0 Sikkim was in the core sample (251 villages).
`latitude`, `longitude`	float	WGS84 (EPSG:4326) coordinates from the geocoding pipeline (Nominatim primary, Bhuvan fallback).
`distance_to_border_km`	float	Distance to the nearest India-relevant Natural Earth Admin-0 boundary segment. The nearest point on the boundary is first located with a planar (lon/lat) nearest-point search (`shapely.ops.nearest_points`), then the distance to that point is measured geodesically (WGS84 ellipsoid, via `pyproj.Geod`) rather than by reprojecting into a single UTM zone -- the study area spans roughly 20 degrees of longitude, so a single-zone projection (the original method, before Development Log Entry 18) is only accurate near its own central meridian. Note this means the *distance measurement* is geodesic but the *nearest-point selection* is not a true ellipsoidal-minimum search; the two only diverge meaningfully for points very close to the antimeridian or poles, neither of which applies here. Added by `compute_border_distance.py`. Present only in the `_with_distance` version, along with

Column	Type	Description
		`nearest_border_country` (which country's segment was actually nearest -- not always China; see Entry 18).

`border_optics_village_results.csv` / `_analyzed.csv` (full-year window) and `_summer.csv` / `_summer_analyzed.csv` (summer-matched window)

One row per village per compositing window, produced by `extract_satellite_data.py` and enriched by `analyze_results.py --window summer` / `analyze_results.py --window full_year` (one consolidated, window-parameterized script as of 2026-09-17; see `archive/README.md`).

Column	Type	Description
`village_id`	int	Join key back to the master village table.
`village`, `district`, `block`, `state`, `is_core_sample`	—	Carried through from the master table for convenience.
`ndbi_before`	float	Mean NDBI over the 500m village buffer, "before" period (2021). Null if the filtered, QA60-cloud-masked Sentinel-2 collection had zero images for that village/window (see `*_image_count` columns below). Note this is a per-image cloud/cirrus bitmask applied before compositing, not a guarantee that every pixel in the 500m buffer was cloud-free in every contributing image.
`ndbi_after`	float	Mean NDBI, "after" period (2025). Same null convention.
`ndbi_change`	float	`ndbi_after - ndbi_before`. Added during analysis, not part of the raw GEE export.
`lights_before`, `lights_after`	float	Mean VIIRS DNB monthly radiance (`avg_rad`) over the same buffer/periods.
`lights_change`	float	`lights_after - lights_before`.
`ndbi_before_image_count`, `ndbi_after_image_count`, `lights_before_image_count`, `lights_after_image_count`	int	Per-metric counts of the Sentinel-2 (NDBI) or VIIRS (lights) images that intersect the village buffer in each period, counted *before* the QA60 cloud mask is applied (so not a count of cloud-free images; a village can have a non-zero count and still a null NDBI if every pixel is masked), present only in the summer-matched treated-village files as of the Entry 22 re-extraction (the extraction script reports NDBI and VIIRS counts separately, since they draw from different collections with different revisit cadences). This is what originally surfaced Sikkim's complete summer-window data loss (BO_Development_Log.md, Entry 5) — later found to be an archive-timing artifact rather than a permanent gap, and resolved as of the complete re-extraction in Entry 22, when all

Column	Type	Description
		core-sample villages, Sikkim included, started carrying non-zero counts. Not present in the full-year treated files; a null `ndbi_before`/`ndbi_after` is the only signal of a missing full-year composite. There is no separate generic `before_image_count`/`after_image_count` pair anywhere in this file — only these four per-metric columns.
`system:index`, `.geo`	—	Google Earth Engine export artifacts (feature index and geometry, GeoJSON-encoded). Not used downstream; harmless to ignore.

Compositing windows, defined precisely

Window	Before	After
Full-year	2021-01-01 to 2022-01-01	2025-01-01 to 2026-01-01
Summer-matched	2021-06-01 to 2021-10-01	2025-06-01 to 2025-10-01

See `src/acquisition/extract_satellite_data.py` for the exact implementation.

`border_optics_control_villages.csv`

One row per non-VVP control village (628 rows from v2.1, all district-verified; 721 rows until v2.0 — deduped by geodesic coordinate proximity (50m) and by the full official priority-village name list; see `BO_Development_Log.md` Entries 14-15, 23-25), assembled via the OpenStreetMap Overpass API from the same 14 districts as the treated core sample, excluding any village on a VVP-I priority list. Produced by `select_control_villages.py`. District membership is checked against a real Nominatim boundary polygon where that lookup succeeds; where it doesn't, the candidate is kept on a rougher bounding-box match instead (see `district_verified` below). A retry/disambiguated-phrasing fix (Development Log Entry 37) was run against live Nominatim/Overpass access and resolved real boundary polygons for two of the three previously-unverified districts (Tawang, Arunachal Pradesh; Pithoragarh, Uttarakhand), dropping the control list from 732 to 721 rows in the process (11 Tawang candidates confirmed genuinely out-of-district once the real polygon was available). **As of the current committed file, that lookup still does not succeed for 93 of the 721 rows, all in Sikkim's North district** — no Nominatim boundary resolves for that district under either query phrasing tried, which appears to be a genuine data gap rather than a network/rate-limit failure — so that one district's fixed effects and cluster assignment still rest on an unverified district label.

Column	Type	Description
`village_id`	int	Control-group-specific integer ID (not comparable to the treated `village_id` values — the two ID spaces are separate).
`village`, `district`, `block`, `state`, `is_core_sample`, `latitude`, `longitude`, `distance_to_border_km`	—	Same meaning as in the master village table.

Column	Type	Description
district_verified	bool	True if this candidate's district membership was confirmed against a real Nominatim administrative-boundary polygon; False if that lookup failed and the row was kept purely because it fell inside the ~65km search bounding box around the district's treated villages instead. Until v2.0, False for 93 of 721 rows (all Sikkim/North); 84 of those lay outside North Sikkim. From v2.1 only True rows are kept (select_control_villages.py drops the rest; the removed rows are in archive/data_processed/v2.0_sikkim_controls/).
village_source	string	Always control (non-VVP) in this file — retained so control and treated rows can be safely concatenated for the DiD panel without losing group identity.

border_optics_control_results.csv (full-year) / border_optics_control_results_summer.csv (summer-matched)

Control-group villages run through the identical extraction pipeline (same 500m buffer, same NDBI/VIIRS definitions, same before/after windows) used for the treated sample. Produced by extract_control_satellite_data.py.

Column	Type	Description
ndbi_before, ndbi_after, ndbi_before_image_count, ndbi_after_image_count	float / int	Same NDBI meaning as the treated village-results files, extracted for control villages — the per-metric image-count columns here (ndbi_*_image_count) are present in BOTH the full-year and summer-matched control files, matching the treated files (which carry these same four per-metric columns in the summer-matched file only; see above).
lights_before, lights_after, lights_before_image_count, lights_after_image_count	float / int	VIIRS radiance equivalents, same per-metric/both-windows image-count columns as the NDBI ones above.

border_optics_did_panel_fullyear.csv / border_optics_did_panel_summer.csv

The treated and control villages reshaped into a two-period panel with a treatment indicator, used to fit the district-fixed-effects Difference-in-Differences model (ndbi ~ treatment + post + treatment×post + district FE, clustered SEs by district). Produced by did_model.py.

Column	Type	Description
village_id	int	Join key (separate ID spaces for treated vs. control, as above).

Column	Type	Description
`district`, `state`	—	Same meaning as in the master village table — carried through so the district fixed effects and cluster-robust standard errors can be fit directly off this panel.
`treatment`	int	`1` for a treated (VVP-I priority) village, `0` for a district-restricted non-VVP control village.
`ndbi`, `lights`	float	The outcome value for this village-period row (one row per village per period, not a before/after delta).
`post`	int	`0` for the "before" period, `1` for the "after" period.
`did_term`	int	`treatment × post` — the interaction term whose coefficient is the DiD estimate.

`border_optics_did_by_district_fullyear.csv` / `border_optics_did_by_district_summer.csv`

Per-district breakdown of the treated-vs-control gap (treated change, control change, and the difference, for each of the 13 districts from v2.1), produced by `did_model.py` alongside the pooled DiD estimate in §5.2 of `BO_Research_Paper.md`, to see whether the pooled gap is spread evenly across districts or concentrated in a few.

Column	Type	Description
`district`	string	District name.
`outcome`	string	`ndbi` or `lights`.
`window`	string	`full_year` or `summer`.
`n_treated`, `n_control`	int	Village counts with valid data in that district/window.
`treated_change`, `control_change`	float	Mean before/after change for each group.
`gap`	float	`treated_change - control_change`.

`border_optics_did_summary_fullyear.json` / `border_optics_did_summary_summer.json`

Top-level keys: `did` (the DiD regression coefficients, confidence intervals, and cluster-robust/HC3 p-values reported in §5.2), and `baseline_balance` (the 2021 Mann-Whitney baseline-imbalance check between treated and control villages, paper v2.0 §5.2 and §6).

`border_optics_multiyear_fullyear.csv` / `border_optics_multiyear_summer.csv`

258 rows — one per geocoded treated village across all four states (Arunachal Pradesh, Uttarakhand, Sikkim and Himachal Pradesh; the core sample is 220 from v2.1), each with NDBI and VIIRS values extracted at all three time points (2021, 2023, 2025), used for the three-point trend extension (§6). Produced by `extract_multiyear_satellite_data.py`.

Column	Type	Description
ndbi_2021, ndbi_2023, ndbi_2025	float	Mean NDBI at each time point.
ndbi_n_2021, ndbi_n_2023, ndbi_n_2025	int	Sentinel-2 image count behind each year's composite.
lights_2021, lights_2023, lights_2025	float	Mean VIIRS radiance at each time point.
lights_n_2021, lights_n_2023, lights_n_2025	int	VIIRS image count behind each year's composite.

border_optics_multiyear_slopes_fullyear.csv / border_optics_multiyear_slopes_summer.csv

220 rows from v2.1 (251 until v2.0) — the core-sample subset of the multiyear files above (Himachal Pradesh's 7 and, from v2.1, Sikkim's 31 non-core villages are dropped here), with a per-village linear trend fit across all three years added by multiyear_trend.py.

Column	Type	Description
ndbi_slope, lights_slope	float	Least-squares slope of the value against year, per village.
ndbi_trend_r2, lights_trend_r2	float	R^2 of that per-village linear fit.

border_optics_multiyear_summary_fullyear.json / border_optics_multiyear_summary_summer.json

A list of summary statistics for the multi-year trend test — the aggregate Wilcoxon signed-rank test on per-village slopes, the village-fixed-effects panel regression, and the individual 2021→2023 / 2023→2025 sub-period tests referenced in §6.

border_optics_buffer250_summer.csv / border_optics_buffer1000_summer.csv

The core-sample villages re-extracted at 250m and 1km buffer radii (summer-matched window only), for the buffer-radius robustness sweep (§6). Same raw extraction columns as border_optics_village_results_summer_analyzed.csv (which serves as the 500m case in this comparison) — ndbi_before/ndbi_after, lights_before/lights_after, and the four per-metric image-count columns — but without that file's derived ndbi_change/lights_change columns, which get computed separately during the buffer-sensitivity analysis rather than stored here. Produced by extract_buffer_sensitivity_data.py.

border_optics_buffer_sensitivity_summary.json

Top-level keys: as_extracted (Wilcoxon results at each radius using all villages with valid data at that radius), coverage_note (valid-village counts at each radius and at all three together — all core villages since the Entry 31 same-day re-pull, 220 from v2.1), and matched_subsample (the same test restricted to villages with valid data at all three radii — as of Entry 22's complete 500m re-extraction, this is the full 251-village core sample at

all three radii, not a smaller subset; the version reported as the robustness result in §6, now null at all three radii).

border_optics_treated_dynamicworld_fullyear.csv / _summer.csv and border_optics_control_dynamicworld_fullyear.csv / _summer.csv

Dynamic World "built" probability, extracted per village/period as one of the three independent-proxy triangulation checks added in Development Log Entries 26-27 (paper v2.0 §5.2). Produced by `src/acquisition/extract_dynamicworld_built.py`. GHSL was considered and explicitly rejected as an alternative before this dataset was chosen, because GHSL's future epochs are modelled rather than observed — see that script's own docstring.

Column	Type	Description
`village_id`, `village`, `district`, `block`, `state`, `is_core_sample`, `latitude`, `longitude`	—	Carried through from the source village table (master for treated, control-villages for control).
`built_before`, `built_after`	float	Mean Dynamic World "built" class probability (0-1) over the same 500m buffer and before/after windows as every other extraction in this study.
`built_before_image_count`, `built_after_image_count`	int	Number of Dynamic World images behind each period's composite — same missing-data convention as `ndbi_before`/`ndbi_after`.
`distance_to_border_km`, `village_source`, `district_verified`	—	Control-file-only columns, same meaning as in `border_optics_control_villages.csv`.

border_optics_treated_sar_fullyear.csv / _summer.csv and border_optics_control_sar_fullyear.csv / _summer.csv

Sentinel-1 SAR backscatter (VV and VH polarizations), the second independent-proxy triangulation check (paper v2.0 §5.2) — SAR is not an optical index, so it is immune to the cloud-masking and cirrus-contamination concerns that motivated the SCL-vs-QA60 comparison for NDBI. Produced by `src/acquisition/extract_sar_backscatter.py`.

Column	Type	Description
`vv_before`, `vv_after`	float	Mean VV-polarization backscatter (dB) over the 500m buffer, before/after periods.
`vh_before`, `vh_after`	float	Mean VH-polarization backscatter (dB), same periods.
`sar_before_image_count`, `sar_after_image_count`	int	Number of Sentinel-1 scenes behind each period's composite.

Both SAR polarizations are null at the control-group DiD stage in every window (paper v2.0 §5.2, `11_triangulation_did.png`) — the triangulation exercise's main negative result, contrasted against Dynamic World's positive one.

`border_optics_treated_pretreatment_fullyear.csv` / `_summer.csv` and `border_optics_control_pretreatment_fullyear.csv` / `_summer.csv`

A single extra pre-treatment year (2019) pulled for both treated and control villages, both windows, so that a genuine parallel-pre-trends placebo test (2019→2021 change, comparing treated vs. control) could be run — the study's existing control-group design otherwise had only one pre-period (2021) and so could only support a baseline-*level* check (§6, Table 5), not a pre-*trend* check. Produced by `src/acquisition/extract_pretreatment_baseline.py`; the placebo DiD itself is run by `src/analysis/pretreatment_placebo_test.py` and reported in §6, Table 5 (Development Log Entry 30).

Column	Type	Description
`ndbi_2019`, `lights_2019`	float	Mean NDBI / VIIRS radiance over the same 500m buffer, 2019-01-01 to 2020-01-01 (full-year) or 2019-06-01 to 2019-10-01 (summer).
`ndbi_2019_image_count`, `lights_2019_image_count`	int	Image counts behind the 2019 composite.

`pretreatment_placebo_test.py` merges these against each group's existing `ndbi_before`/`lights_before` (i.e. the 2021 values already on disk) to construct the 2019→2021 placebo change, rather than storing that change in these files directly.

`border_optics_treated_dw_pretreatment_fullyear.csv` / `_summer.csv` and `border_optics_control_dw_pretreatment_fullyear.csv` / `_summer.csv`

The Dynamic World analogue of the pretreatment-baseline files above, added because paper v2.0 §5.2's triangulation shows Dynamic World disagreeing with NDBI on the control-group DiD (DW: small but significant; NDBI: null) — this asks whether that DW gap was already present before treatment could apply, not just after. Produced by `src/acquisition/extract_dw_pretreatment_baseline.py`. Requires live GEE; not run in my analysis environment that added this script.

Column	Type	Description
`built_2019`	float	Mean Dynamic World "built" class probability (0-1) over the same 500m buffer, 2019-01-01 to 2020-01-01 (full-year) or 2019-06-01 to 2019-10-01 (summer).
`built_2019_image_count`	int	Number of Dynamic World images behind the 2019 composite.

`src/analysis/dw_pretreatment_placebo_test.py` merges these against each group's existing `built_before` (the 2021 value already in `border_optics_{treated,control}_dynamicworld_{fullyear,summer}.csv`) to construct the 2019→2021 DW placebo change.

`border_optics_treated_building_footprints.csv` / `border_optics_control_building_footprints.csv`

Independent, non-satellite-index building counts from Google's Open Buildings dataset (`GOOGLE/Research/open-buildings/v3/polygons`, confidence ≥ 0.75), used to validate each proxy's *current* (roughly 2025-era) built-up reading (§5.4, Figure 6) and to check whether the §5.4 villages with documented new facilities' real additions show up as detected structures. Produced by `src/acquisition/extract_building_footprints.py`. Single-vintage only — Open Buildings has no historical epoch to difference, unlike every before/after extraction elsewhere in this study; this was discovered and documented in the script rather than assumed to exist.
`src/analysis/building_footprint_validation.py`'s published correlations (paper v2.0 Figure 6A) (rho=0.399 NDBI, rho=0.478 lights, rho=0.808 Dynamic World) are computed from the **treated-group file only** — the control-group file exists on disk (721/721 villages as of Development Log Entry 44) but isn't currently read by that script or any other; it's kept extracted and current-list-matched for parity with every other control-group extraction in this study, and would be the input if a treated-vs-control footprint-density comparison is added later.

Column	Type	Description
`building_count`	int	Number of high-confidence building footprint polygons intersecting the 500m buffer.
`building_total_area_m2`	float	Summed full footprint area (m^2) of those polygons — **not clipped to the 500m buffer**: a polygon that straddles the buffer edge contributes its whole area, not just the part inside the buffer, so this is an upper bound rather than an exact in-buffer area. `building_count` is unaffected and is the field used in the published building-count correlations.
`building_mean_confidence`	float	Mean detection confidence across the polygons counted (null if `building_count` is 0).

`outputs/triangulation_results.json`

Per-proxy (SAR VV, SAR VH, Dynamic World "built") replication of the study's own two core tests — the treated-only before/after Wilcoxon (`h1_treated_only`) and the control-group DiD (`h4_control_group_did`) — run on each proxy exactly as `did_model.py` runs them on NDBI, so the four proxies are apples-to-apples comparable at the test-design level (not at the coefficient-magnitude level — each proxy has its own units/scale, which is why `11_triangulation_did.png` gives each proxy its own x-axis). Produced by `src/analysis/triangulation_analysis.py`. Top-level keys are `{proxy}_{window}` (e.g. `dynamicworld_built_summer`, `sar_vv_full_year`). The headline finding (paper v2.0 §5.2): Dynamic World's control-group DiD is significant in both windows (p=0.0159 full-year, p=0.0006 summer with the normal reference stored in this file; t with 13 df gives 0.031 and 0.004, wild cluster bootstrap 0.041 and 0.008 — see

dynamic_world_robustness_results.json) while NDBI and both SAR polarizations are null in every window — one proxy corroborates the primary NDBI-null result's own DiD design finding *something*, while two others corroborate the null itself.

outputs/building_footprint_validation.json

Top-level keys building_count_vs_ndbi_after, building_count_vs_lights_after, building_count_vs_built_after — Spearman correlation (rho, p, n) between each proxy's 2025-era ("after") level and the independent Open Buildings count, i.e. Panel A of 12_footprint_validation.png (paper Figure 6). Dynamic World correlates far more strongly with building counts (ρ=0.808) than NDBI (ρ=0.399) or night-lights (ρ=0.478) do; this is a cross-sectional level correlation and says nothing about detecting change. case_study_villages is a 3-entry list (Kaho, Walong, Musai, the Anjaw villages with facilities inaugurated in October 2023) with building_count, building_total_area_m2, and each proxy's _after level. Panel B (v2.0) plots each village's full-year 2021→2025 change as a percentile of all 251 core villages' change for that proxy, computed directly from the extraction files. Note (v2.0): the source news reports do not describe these villages' works as VVP-funded; see paper §5.4.

outputs/h3_robustness_results.json

The H3 (border-proximity) stress test (paper v2.0 §5.5, Development Log Entry 32) applied to all four outcome/window combinations, not just the summer-NDBI result the main text originally flagged as open. Produced by src/analysis/h3_border_proximity_robustness.py. Top-level keys are summer_ndbi_change, full_year_lights_change (the two significant H3 results), and full_year_ndbi_change, summer_lights_change (the two already-null ones, checked anyway for completeness). Each key holds full_sample (the headline rho/p/n), leave_one_district_out (one refit per district, each dropping one district, with n_significant, n_same_sign and n_reruns summary counts; 14 refits until v2.0, 13 from v2.1), and randomization_inference (2,000-permutation null distribution of rho, giving p_randomization). Both significant H3 results survive all 14 leave-one-out refits and randomization inference; the summer-NDBI result's leave-one-out rho range is 0.183-0.337.

outputs/spatial_moran_h3_correction_results.json

Two top-level keys. moran_reproduction reproduces v1.7 §6.11's own published Moran's I values (k=8 nearest-neighbor haversine weights, 999-permutation significance) as an internal-consistency check before trusting the same weight matrix for the spatial correction below — both reproduced values matched the published claim to 3 decimal places. h3_spatial_correction (one entry per significant H3 result) reports the SAR(1)-calibrated spatial-permutation-null p-value: calibrated_sar_phi is the autoregressive parameter chosen so simulated spatial fields' mean Moran's I matches the real outcome's; spatially_corrected_p is the fraction of 2,000 such simulated fields whose correlation with the (fixed, real) distance-to-border values is at least as extreme as the observed one. Both H3 findings survive: summer-NDBI naive p=2.6e-06 → spatially-corrected p=0.0295; full-year-lights naive p=5.5e-05 → spatially-corrected p=0.0035. Produced by src/analysis/spatial_moran_and_h3_correction.py.

outputs/wild_cluster_bootstrap_results.json

A 4-row list (one row per outcome × window), addressing the "only 14 district clusters" limitation flagged in paper v2.0 §4.1 and Limitations/ANALYSIS_FREEZE.md — with only 14 clusters, asymptotic cluster-robust SEs are below the usual 30-40+ recommended minimum, so this re-derives significance via full enumeration of all

2^14=16,384 Rademacher sign-flip combinations (Cameron, Gelbach & Miller 2008 restricted/WCR variant) rather than relying on the asymptotic approximation. Produced by `src/analysis/wild_cluster_bootstrap.py` (manual numpy OLS + cluster-robust SE, validated against `did_model.py`'s own published coefficients to ~10 significant figures before being trusted here, since `statsmodels` could not be installed in my analysis environment that built this script). Columns: `real_did_coef`/`real_cluster_se`/`real_t` (the original estimate), `n_bootstrap_draws`/`n_combinations_total` (always 16384, exact enumeration not Monte Carlo), `wild_cluster_bootstrap_p` (the result), `boot_t_mean`/`boot_t_sd` (bootstrap null-distribution diagnostics). All four DiD coefficients remain non-significant under this stricter test (p=0.354 to p=0.680 on the current 721-village list; p=0.324 to p=0.495 on the earlier 732-village list) — consistent with, not contradicting, the asymptotic cluster-robust result already reported in §5.2.

`outputs/dw_sar_ndbi_village_level_results.json`

Village-level (not aggregate-significance) Spearman cross-checks between the three built-up proxies' before→after *changes*, addressing whether NDBI, SAR, and Dynamic World agree in *direction* at the individual-village level, independent of whether any of them clears significance in aggregate (paper v2.0 §5.2). Produced by `src/analysis/dw_sar_ndbi_village_level_check.py`. Top-level keys `full_year`/`summer`, each with four pairwise comparisons (`built_change_vs_vv_change`, `built_change_vs_vh_change`, `built_change_vs_ndbi_change`, `vv_change_vs_ndbi_change`) giving `rho`, `p`, `n`, and `same_sign_pct` (the percentage of villages where the two proxies' changes share a sign), plus `built_change_by_state` (mean Dynamic World change per state). Same-sign agreement across proxy pairs sits in the 31-63% range — near chance for a binary sign match — sharpening rather than resolving the underlying disagreement between proxies that paper v2.0 §5.2's aggregate-level triangulation result raises.

`outputs/pretreatment_placebo_summary_fullyear.json` / `_summer.json`

The 2019→2021 placebo DiD results (§6, Table 5, Development Log Entries 30, 42) — same district-fixed-effects, cluster-robust-SE specification as `did_model.py`'s real 2021→2025 DiD, run instead on the genuinely pre-treatment 2019-to-2021 change. Produced by `src/analysis/pretreatment_placebo_test.py`. Three of four outcome/window combinations are clean nulls (full-year NDBI p=0.73497, full-year lights p=0.75898, summer lights p=0.52021); summer NDBI is the flagged exception (placebo coefficient +0.01173, p=0.07421 — borderline, and reported as such rather than rounded up to "passes," with the raw unadjusted Mann-Whitney comparison for the same combination sharply significant at p=0.00003). These are the district-verification-corrected, 721-village figures (Development Log Entry 42); the immediately preceding 732-village version had full-year NDBI p=0.864, full-year lights p=0.755, summer lights p=0.968, and summer NDBI p=0.064 (raw p=0.00001) — the same borderline pattern, no reversal.

`outputs/dw_pretreatment_placebo_summary_fullyear.json` / `_summer.json`

The Dynamic World analogue of `outputs/pretreatment_placebo_summary_*.json` — same district-fixed-effects, cluster-robust-SE placebo-DiD specification, run on the 2019→2021 change in DW "built" probability instead of NDBI/lights. Produced by `src/analysis/dw_pretreatment_placebo_test.py`. Single top-level key `built` (one outcome, since Dynamic World has only the one band), with the same fields as `pretreatment_placebo_test.py`'s per-outcome result (`placebo_did_coef`, `placebo_did_se`,

`placebo_did_p`, confidence interval, Mann-Whitney p). Run against live GEE at Development Log Entry 43: full-year (n=962, treated=247/control=715, coef=+0.00009, SE=0.00112, p=0.93406, Mann-Whitney p=0.65024) and summer (n=652, treated=161/control=491, coef=+0.00207, SE=0.00154, p=0.17869, Mann-Whitney p=0.10256) — both clean nulls, unlike NDBI's own borderline summer exception; see `BO_Research_Paper.md` §5.2 (v2.0).

`outputs/robustness_extended_results.json`

Two additional robustness checks beyond the ones with their own dedicated files above. `leave_one_district_out` re-fits the *real* 2021→2025 DiD (not the H3 proximity test) dropping one district at a time, for both NDBI and lights — `n_significant: 0` for both, i.e. the real DiD result stays null under every leave-one-out refit, consistent with (not contradicted by) it already being null in the full sample. `randomization_inference` reruns the real DiD's treatment assignment as a 2,000-permutation randomization-inference null (`p_randomization`) as a non-parametric cross-check on the cluster-robust asymptotic p-value. A third key, `log_viirs`, checks whether a log1p transform of the VIIRS radiance outcome changes any significance verdict (it does not). Produced by `src/analysis/extended_robustness_checks.py`.

`outputs/leave_one_district_out_results.csv` / `outputs/holm_correction_results.csv`

Tabular (CSV) companions to two of the JSON robustness outputs above, in the flat per-row format used by the dashboard's Statistical Validation page: `leave_one_district_out_results.csv` is the row-per-dropped-district version of `robustness_extended_results.json`'s `leave_one_district_out` key; `holm_correction_results.csv` is the Holm-Bonferroni multiple-comparisons correction (`src/analysis/holm_correction.py`) applied specifically across the 8 pre-specified H1/H3 headline p-values (NDBI and lights, full-year and summer, treated-only and border-proximity) — see `ANALYSIS_FREEZE.md`'s "Multiple-testing family" section for the exact list. It is NOT applied across every test this study reports: the control-group DiD (H4), buffer-radius sweep, leave-one-district-out, randomization inference, and multi-year checks are robustness checks on those 8 headline results, not additional members of the corrected family.

`outputs/buffer_overlap_sensitivity_results.json`

Added in Development Log Entry 48. The control-group contamination fix excludes candidates within 50m of a treated village, but extraction uses a 500m buffer, so a control village less than 1km from a treated village shares pixels with it. This file records how many controls are in that position (`n_control_overlapping` = 82 of 721, `n_control_within_200m` = 5) and re-runs the primary district-FE, cluster-robust DiD without them (`did_excluding_overlap`, one row per outcome × window: `n_treated`, `n_control`, `did_coef`, `did_se`, `did_p`). All four stay null (p = 0.322 / 0.511 / 0.631 / 0.783 for full-year NDBI / full-year lights / summer NDBI / summer lights). Produced by `src/analysis/buffer_overlap_sensitivity.py`.

`border_optics_village_results_summer_sclmask.csv` (superseded, kept for the record only)

The original SCL re-extraction, buggy: `SCL_CLEAR_CLASSES` included class 11, which is snow/ice, not clear ground. Left on disk untouched as the pre-fix record (caught by external review, Development Log Entry 34) — do not use this file for anything; use the corrected file below instead.

border_optics_village_results_summer_sclmask_corrected.csv / border_optics_village_results_summer_sclmask_strict.csv

Summer-window NDBI, treated core sample only, re-extracted with an SCL (Scene Classification Layer) per-pixel cloud mask in place of the primary pipeline's QA60 bitmask, as a cross-check on cloud-masking sensitivity (§6, Table 5, Development Log Entries 33-34). `corrected` uses clear classes 2/4/5/6 (dark area, vegetation, bare soils, water); `strict` uses 4/5/6 (drops dark-area pixels too, as a second cross-check). Both exclude class 11 (snow/ice) — the earlier file above did not. Produced by `src/acquisition/extract_scl_cloud_mask_ndbi.py`.

Column	Type	Description
`ndbi_scl_before`, `ndbi_scl_after`	float	Mean NDBI over the same 500m buffer and before/after periods as the primary extraction, but built from an SCL-masked composite. Null if the mask left zero clear images for that village/period — the `corrected` variant returns valid data for 251/258 villages, matching QA60's own 251/258 coverage almost exactly (no longer skewed toward Arunachal Pradesh, unlike the pre-fix file's 200/258).
`ndbi_scl_before_image_count`, `ndbi_scl_after_image_count`	int	Number of images behind each period's SCL-masked composite.

outputs/scl_vs_qa60_comparison.json

The result of comparing the corrected SCL file above against the primary QA60-masked summer NDBI extraction (`border_optics_village_results_summer_analyzed.csv`), via `src/analysis/scl_vs_qa60_comparison.py`. Reports `n_qa60_valid`/`qa60_mean_change`/`qa60_wilcoxon_p_full` (the primary, full n=251 result), `n_scl_valid`/`scl_mean_change`/`scl_wilcoxon_p` (the corrected SCL-masked result), `qa60_matched_subsample_n`/`_mean_change`/`_wilcoxon_p` (QA60 recomputed on the identical SCL-valid villages — now the same 251 villages as the full sample, since coverage matches), `village_level_agreement_rho`/`_p` (Spearman correlation between the two masks' NDBI-change values), `masks_agree_on_significance` (bool), and `coverage_diff`. Current result, post-fix: both masks null (QA60 p=0.436, SCL p=0.288), `masks_agree_on_significance=true` — the pre-fix run of this same script (buggy SCL classes, n=200) had reported QA60 null against SCL significant (p=0.0000045); that was the bug, not a real disagreement. The `strict` variant (classes `[4,5,6]`) agrees too, checked separately and not (yet) written into this JSON's own fields: n=251, mean change +0.00187, Wilcoxon p=0.17501, p=0.736 against QA60. Note: the version of this file first committed after the script fix still had the *old* hardcoded narrative text in its `coverage_note` field even though its numeric fields were already correct — caught by external review, regenerated, and re-synced; if you're diffing an older copy against this description, check the `coverage_note` field specifically.

Statuses used in the geocoding pipeline (data/processed/<state>_geocoded.csv)

`geocode_status` value	Meaning
`matched`	Nominatim matched on the full query (habitation + block + district + state).
`matched (fallback query, no block)`	Nominatim matched only after dropping the block name.
`matched via Bhuvan (district-verified)`	Bhuvan matched, and the returned district matched the expected district.
`NO MATCH — Bhuvan also failed or district mismatch`	Bhuvan either found nothing or returned a same-named village in the wrong district (discarded, not accepted).
`NO MATCH — not in OSM (tried full + simplified query)`	Nominatim found nothing under either query form; retried against Bhuvan in the fallback pass.
`EXCLUDED — forest block, not an inhabited settlement`	Entry was a forest survey compartment, not a village — excluded before geocoding was attempted.
`FAILED AFTER RETRIES — network/timeout issue, retry later`	Transient network failure; re-running the script retries these rows specifically.

Files added in v2.0 (Development Log Entry 49)

`data/processed/border_optics_lac_distance.csv`

One row per treated (258) and control (628 from v2.1; 721 until v2.0) village, from `src/analysis/compute_lac_distance.py`.

Column	Meaning
`group`	`treated` or `control`
`village_id`, `village`, `district`, `state`	as in the source village list (IDs are unique *within* a group only)
`distance_to_border_km`	committed distance to the nearest India boundary line (any neighbour)
`distance_to_border_km_recomputed`	same distance recomputed by `border_geometry.py`; must match within 1 m (tested)
`nearest_border_country_recomputed`	neighbouring country of the nearest boundary segment
`distance_to_china_km`	geodesic distance to the nearest India–China boundary segment (Natural Earth; cartographic, not authoritative)

`outputs/h3_within_district_results.json`

From `h3_within_district.py`. Keys `window|outcome|distance_measure`. Fields: `pooled_rho`/`pooled_p` (Spearman, all villages); `within_district_rank_corr` (stratified Spearman: ranks within district, centred per district) and `within_district_perm_p` (2,000 within-district permutations, seed 42); `district_fe_rank_slope`, `district_fe_se`, `district_fe_p_t` (rank regression with district dummies, cluster-robust, t with 13 df); `between_district_rho_n14`; `per_district_rho_n_ge_8`.

`outputs/measurement_sensitivity_results.json`

From `measurement_sensitivity.py`. Buffer area, median treated building area and share, number of building-free buffers, and per outcome (`ndbi_full_year`, `ndbi_summer`, `dw_built_full_year`, `dw_built_summer`): `contrast_slope_per_unit_building_share` (index change per unit building cover, cross-sectional, district FE, clustered SE); `implied_buffer_mean_change` for +50% / +100% of the median village's building area; the DiD (`did`, with t(13) 90%/95% CIs); the DiD and its 95% CI expressed as building-equivalent m^2; TOST equivalence against ± a doubling; the doubling as a fraction of the H1/H4 MDEs (NDBI only).

`outputs/dynamic_world_robustness_results.json`

From `dynamic_world_robustness.py`. Dynamic World DiD per window with t p (G − 1 df), exact wild cluster bootstrap p, leave-one-district-out summary, and variants restricted to summer-valid villages, and without the 10 qualifier-matched controls.

`outputs/sample_qa_sensitivity_results.json`

From `sample_qa_sensitivity.py`. Keys `window|outcome|variant`; variants: `baseline`, `dedup_identical_coordinates`, `drop_zero_building_buffers`, `drop_joshimath`, `drop_suffix_matched_controls`, `all_combined`, and (full year) `python_pipeline_treated_values`. Fields: sample sizes, mean/median change, H1 Wilcoxon p (one- and two-sided), DiD coefficient and p (t13 and z).

`outputs/settlement_masked_did_results.json` (created once the footprint-zone extraction has been run)

From `settlement_masked_did.py`, reading `border_optics_{treated,control}_settlement_masked_{full_year,summer}.csv` produced by `src/acquisition/extract_settlement_masked.py`. Outcome columns per year: `ndbi_zone`, `ndbi_n`, `dw_built_zone`, `dw_built_area_m2`, `dw_n`, `viirs_cf`, `viirs_n`, plus `zone_area_m2` and `n_footprints`. See that script's docstring for definitions.

`outputs/s2_archive_ingestion_check.csv` (created once `check_s2_archive_ingestion.py` has been run)

Per Sikkim village and summer window: scene count and how many scenes were ingested into Earth Engine after the original July 2026 extraction.

Changes to existing files in v2.0

- `outputs/leave_one_district_out_results.csv`: new first column `window` (all rows are summer-window reruns; the column was implicit before).
- Figures `01`–`12` and `static_map_renders/*` regenerated with the v2.0 scripts (no hard-coded figure numbers; see each script's docstring).

Changes in v2.1 (7 November 2026)

- **Sample.** `is_core_sample` is `False` for Sikkim in every treated file; Sikkim rows were removed from every `border_optics_control_*.csv` file (kept in

archive/data_processed/v2.0_sikkim_controls/). Core sample: 220 treated, 628 control villages, 13 districts.

- **Model.** Every DiD (did_model.py, did_numpy.py, triangulation_analysis.py, buffer_overlap_sensitivity.py, wild_cluster_bootstrap.py, extended_robustness_checks.py) includes district-by-period effects. In border_optics_did_summary_*.json, did_coef and did_coef_nofe_hc3 therefore now differ.
- **Renamed keys** (the old names had the cluster count built in): p_t13 → p_t in outputs/triangulation_holm_results.json; did_p_t → did_p_t in outputs/sample_qa_sensitivity_results.json; n_significant_t13 → n_significant_t in outputs/dynamic_world_robustness_results.json; n_significant_of_14 → n_significant (plus n_reruns) in outputs/robustness_extended_results.json and outputs/h3_robustness_results.json (n_same_sign_of_14 → n_same_sign); district_fe_p_t13 → district_fe_p_t and between_district_rho_n14 → between_district_rho in outputs/h3_within_district_results.json.
- **Removed.** full_year_without_sikkim in outputs/dynamic_world_robustness_results.json (no longer needed). Figure 02_state_change_vs_budget.png moved to archive/figures/02_state_change_vs_budget_v2.0.png (only two states remain).
- **Figures** 01, 03–12 and static_map_renders/* regenerated; Figure 11 now marks Holm-adjusted significance.